

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.impute import SimpleImputer
```

```
boston = pd.read_csv("Boston.csv")
```

```
print("-----Dataframe Info-----")
print(boston.info())
print("\n")
```

```
-----Dataframe Info-----
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   CRIM        486 non-null    float64
1   ZN          486 non-null    float64
2   INDUS       486 non-null    float64
3   CHAS        486 non-null    float64
4   NOX         506 non-null    float64
5   RM          506 non-null    float64
6   AGE         486 non-null    float64
7   DIS         506 non-null    float64
8   RAD         506 non-null    int64
9   TAX         506 non-null    int64
10  PTRATIO     506 non-null    float64
11  B           506 non-null    float64
12  LSTAT       486 non-null    float64
13  MEDV        506 non-null    float64
dtypes: float64(12), int64(2)
memory usage: 55.5 KB
None
```

```
print("-----Dataframe Describe-----")
print(boston.describe())
print("\n")
```

```
-----Dataframe Describe-----
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	\
count	486.000000	486.000000	486.000000	486.000000	506.000000	506.000000	
mean	3.611874	11.211934	11.083992	0.069959	0.554695	6.284634	
std	8.720192	23.388876	6.835896	0.255340	0.115878	0.702617	
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	

25%	0.081900	0.000000	5.190000	0.000000	0.449000	5.885500
50%	0.253715	0.000000	9.690000	0.000000	0.538000	6.208500
75%	3.560263	12.500000	18.100000	0.000000	0.624000	6.623500
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000

	AGE	DIS	RAD	TAX	PTRATIO	B \
count	486.000000	506.000000	506.000000	506.000000	506.000000	506.000000
mean	68.518519	3.795043	9.549407	408.237154	18.455534	356.674032
std	27.999513	2.105710	8.707259	168.537116	2.164946	91.294864
min	2.900000	1.129600	1.000000	187.000000	12.600000	0.320000
25%	45.175000	2.100175	4.000000	279.000000	17.400000	375.377500
50%	76.800000	3.207450	5.000000	330.000000	19.050000	391.440000
75%	93.975000	5.188425	24.000000	666.000000	20.200000	396.225000
max	100.000000	12.126500	24.000000	711.000000	22.000000	396.900000

	LSTAT	MEDV
count	486.000000	506.000000
mean	12.715432	22.532806
std	7.155871	9.197104
min	1.730000	5.000000
25%	7.125000	17.025000
50%	11.430000	21.200000
75%	16.955000	25.000000
max	37.970000	50.000000

```
print("-----Dataframe 5 Rows-----")
print(boston.head())
print("\n")
```

```
-----Dataframe 5 Rows-----
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	\
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296	15.3	
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242	17.8	
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242	17.8	
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222	18.7	
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3	222	18.7	

	B	LSTAT	MEDV
0	396.90	4.98	24.0
1	396.90	9.14	21.6
2	392.83	4.03	34.7
3	394.63	2.94	33.4
4	396.90	NaN	36.2

```
print("-----Dataframe Columns List-----")
print(boston.columns)
print("\n")
```

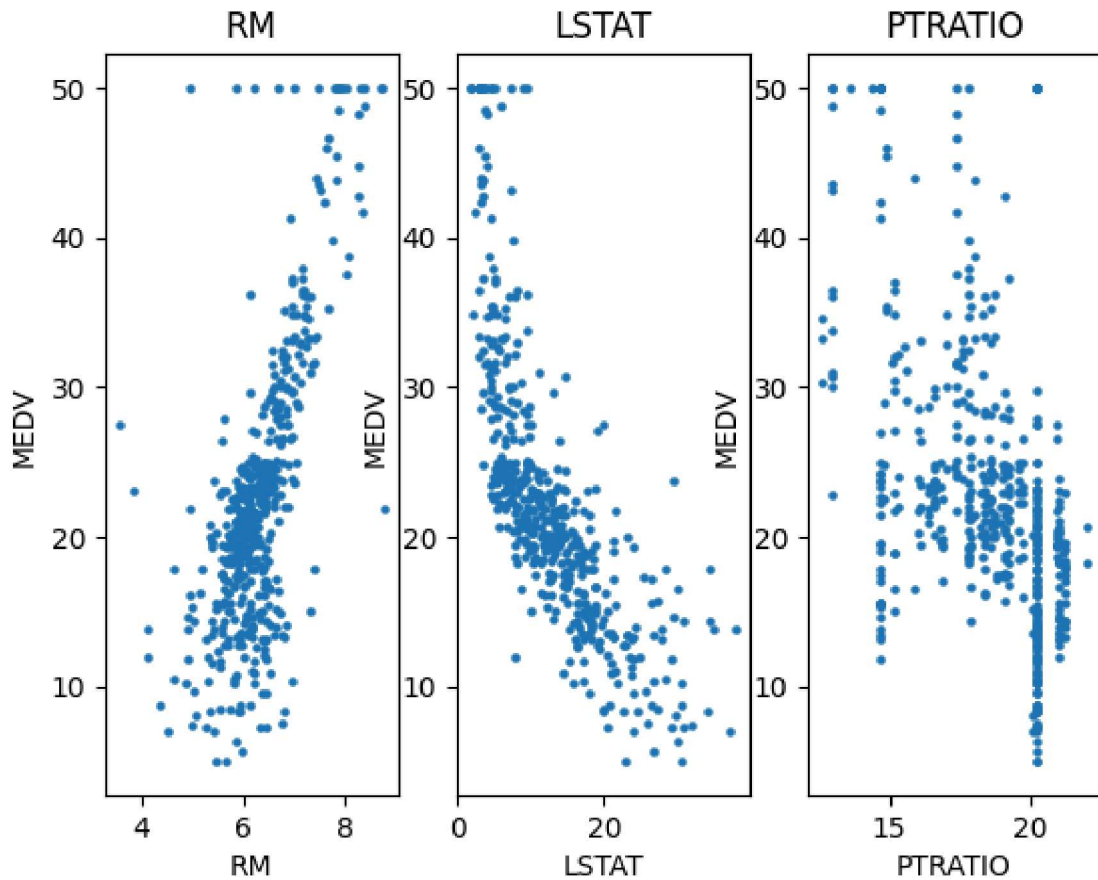
```
-----Dataframe Columns List-----  
Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',  
      'PTRATIO', 'B', 'LSTAT', 'MEDV'],  
      dtype='object')
```

```
X = boston[['RM', 'LSTAT', 'PTRATIO']]  
y = boston['MEDV']
```

```
print("-----Splitting data into training and test sets-----")  
# Splitting the data into training and testing sets  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random  
print("X_train shape:", X_train.shape)  
print("X_test shape:", X_test.shape)  
print("y_train shape:", y_train.shape)  
print("y_test shape:", y_test.shape)  
print("\n")
```

```
-----Splitting data into training and test sets-----  
X_train shape: (404, 3)  
X_test shape: (102, 3)  
y_train shape: (404,)  
y_test shape: (102,)
```

```
imputer = SimpleImputer(strategy='mean')  
X_train = imputer.fit_transform(X_train)  
X_test = imputer.transform(X_test)  
  
for i, feature in enumerate(X.columns):  
    plt.subplot(1, 3, i + 1)  
    plt.scatter(X[feature], y, marker='o', s=5)  
    plt.title(feature)  
    plt.xlabel(feature)  
    plt.ylabel('MEDV')
```



```
plt.tight_layout()
plt.show()
```

<Figure size 640x480 with 0 Axes>

Creating a linear regression model

Training the model

Making predictions

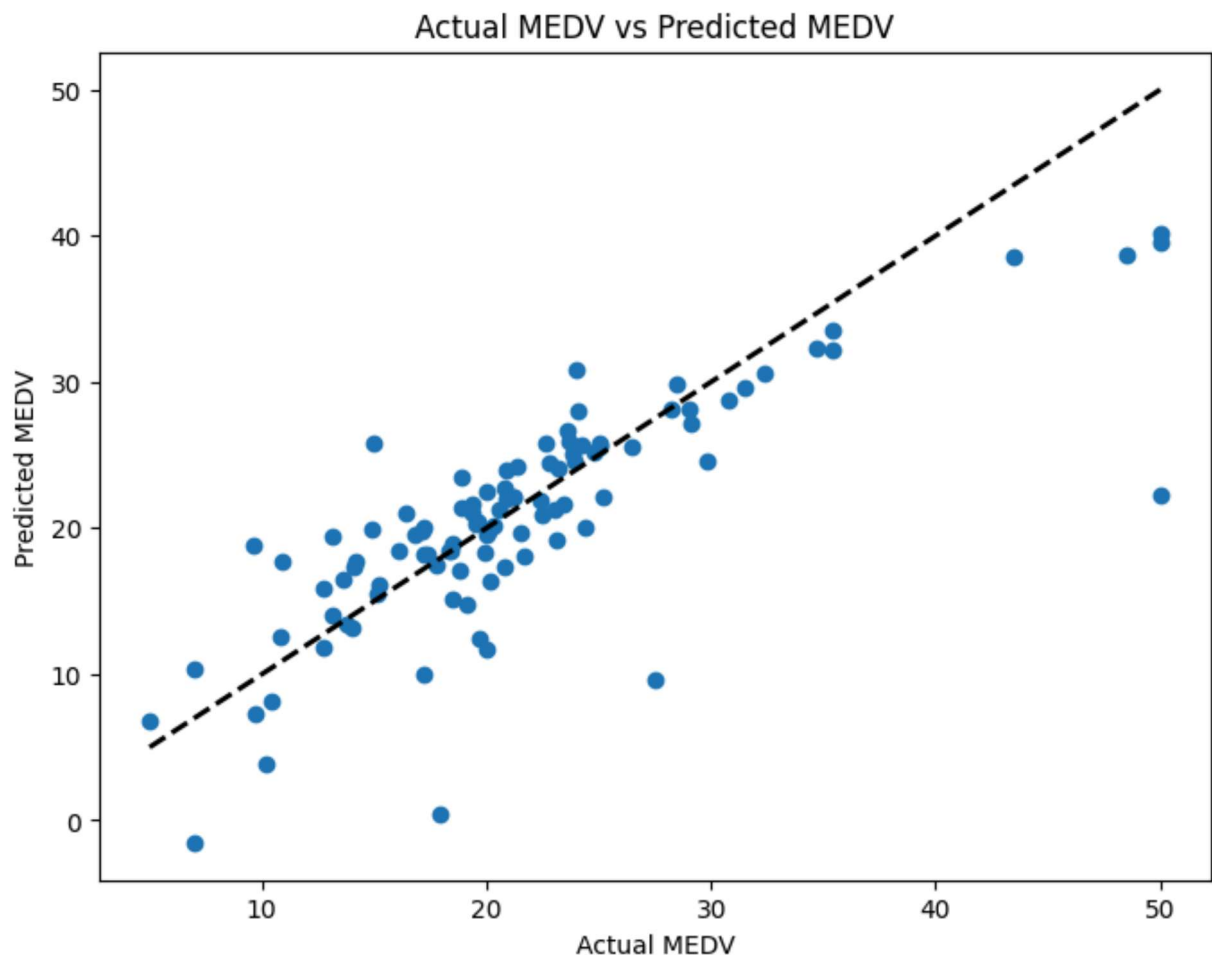
```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

◆ Gemini

```
print("-----Visualization after fitting model-----")
# Visualization after fitting the model
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--')
plt.xlabel('Actual MEDV')
plt.ylabel('Predicted MEDV')
plt.title('Actual MEDV vs Predicted MEDV')
```

```
plt.show()
print("\n")
```

-----Visualization after fitting model-----



```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("-----Evaluation Result-----")
print("Mean Squared Error:", mse)
print("R^2 Score:", r2)
print('\n')
```

```
-----Evaluation Result-----
Mean Squared Error: 27.505630947105466
R^2 Score: 0.6249255150661951
```